

Convolutional Neural Networks

Phuong T. Nguyen



Dipartimento di Ingegneria e Scienze
dell'Informazione e Matematica

Università degli Studi dell'Aquila

Agenda

- Hyperparameters: mini-batch, epoch
- How convolutional neural networks learn from data
- Basic components: convolution, pooling, fully connected layers.
- Activation functions
- Overfitting and underfitting
- Dropout

Hyperparameters

- A sample/instance is a row of data.
- Batch size: a number of samples passed through the model before updating the weights.
- After one batch, the predicted labels and the real ones are compared to compute the error.
- Epoch: the number of times that the entire training data is used to train the model.

■ Mini-batch

- A subset of the training data that is used for a single update of the model's parameters (weights and biases).
- Instead of presenting the entire training dataset to the model at once, it is divided into smaller, equal parts.
- Efficiency: Training on the entire dataset at once can be computationally expensive and memory-intensive. Mini-batches allow for more efficient processing and utilize parallelism in hardware like GPUs.

■ Mini-batch

- **Stochasticity:** Mini-batches introduce a degree of randomness in the parameter updates, helping the model escape local minima and explore a broader optimization landscape.
- This is especially important in stochastic gradient descent (SGD).
- **Generalization:** Mini-batches help the model generalize better by preventing it from memorizing the entire dataset. Instead, the model learns from diverse examples in each mini-batch.

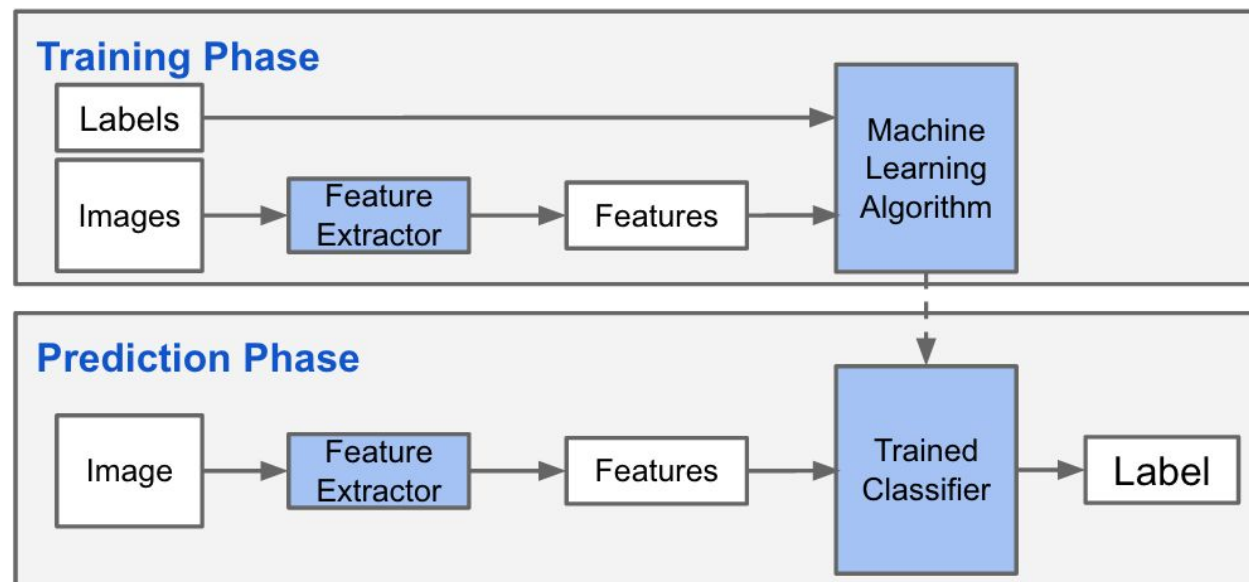
Epoch

- A complete pass through the entire training dataset.
- During one epoch, the model processes and updates its parameters based on each mini-batch, iterating through all the mini-batches in the dataset.
- After each epoch, the model's parameters have been updated multiple times based on different mini-batches, and the model has the opportunity to learn from the entire dataset.
- The training process involves multiple epochs, allowing the model to learn from different mini-batches of data and improve its performance over time.

Training

- The training dataset is randomly shuffled to ensure that each epoch presents the data in a different order, helping with generalization.
- The data is divided into mini-batches.
- The model processes each mini-batch, computes the loss, and updates its parameters using an optimization algorithm, typically gradient descent.
- The process is repeated for all mini-batches in the dataset.

Training and testing



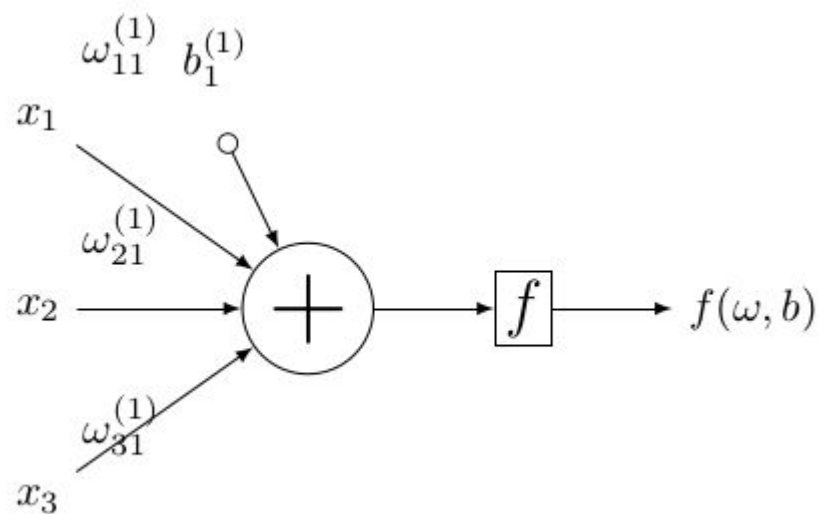
Machine Learning Phases

Image source:

<http://adilmoujahid.com/posts/2016/06/introduction-deep-learning-python-caffe/>

- Training (learning): extract meaningful patterns from data to build a model.
- Testing (deployment): using the model to predict unknown data.

Neural Network: Perceptron



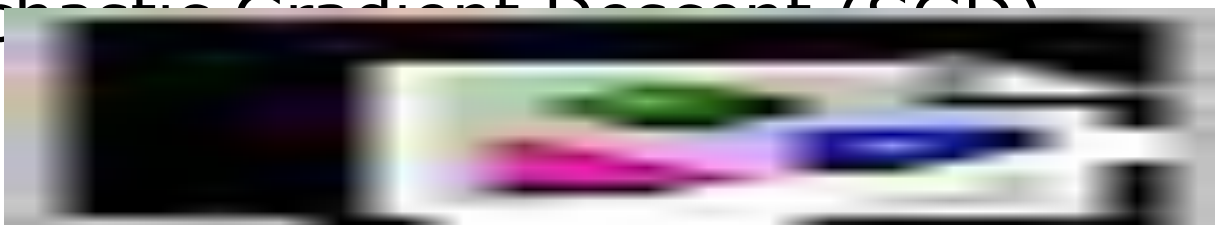
(a) A simple perceptron with 3 inputs

$$output = f\left(\sum_{i=1}^3 \omega_{i1}^{(1)} \cdot x_i + b^{(1)}\right)$$

- A decision making unit, taking into consideration different inputs.

Learning

- Find a set of **weights and biases** (w, b) that minimize the cost function between the **predicted values** and the **real ones** using Stochastic Gradient Descent (SGD)



- Δy is a linear function of Δw and Δb

$$\Delta y \approx \sum_j \frac{\partial y}{\partial w_j} \Delta w_j + \frac{\partial y}{\partial b} \Delta b$$

Learning ability

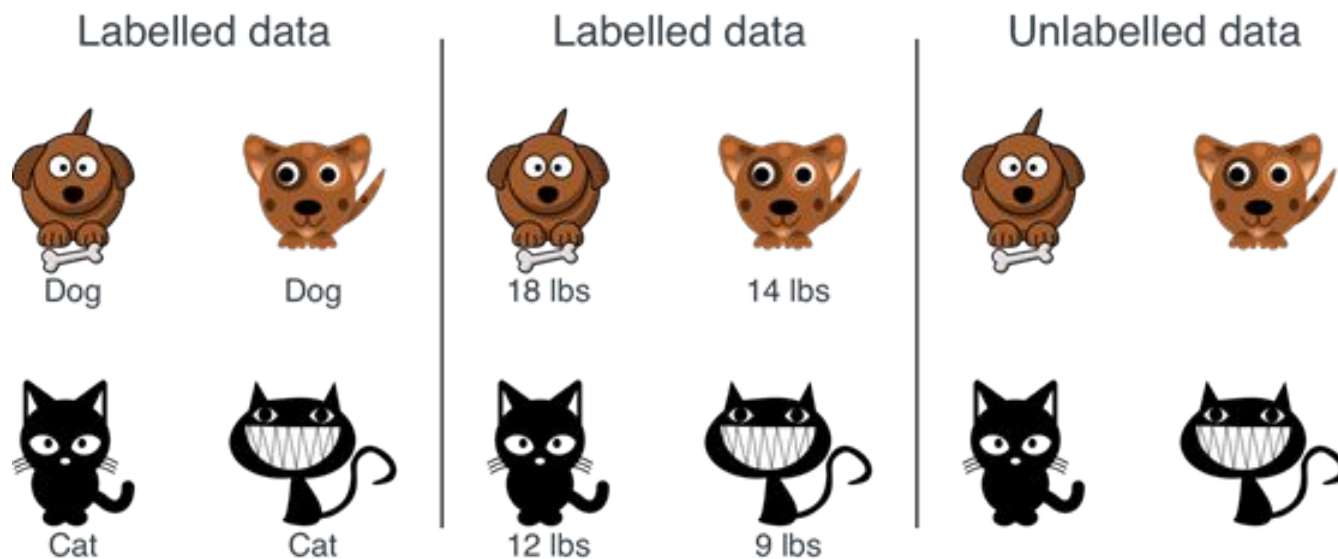


Image source: <https://bit.ly/3RIBPUJ>

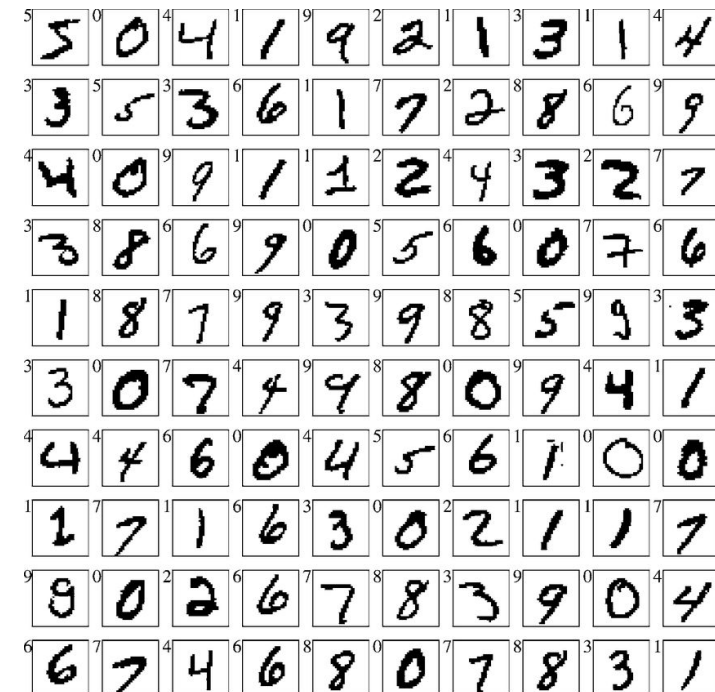


Image source: <https://link.springer.com/article/10.1023/A:1012454411458>

- Neural networks learn by tuning their parameters to build a function to describe the input data.
- The training data needs to be properly classified by humans.
- The more data is fed, the better neural networks can learn.

Derivative and gradient descent

- **Derivative** is used to find the position where a function (with a few variables) reaches the minimum.
- When the function becomes complicated with many variables, and the change can happen with every variable, we need **gradient descent**.
- Gradient descent is used to training data to iteratively update/improve an ML model.

— The need for CNNs

- Ordinary neural networks connect all the neurons of a layer to the next layer.
- The number of weights and biases increases exponentially with the number of layers.
- Feedforward neural networks are well-suited for structured data like tabular data, time series, and data where each input has a fixed number of features. They are not designed to handle grid-like data (e.g., images), or sequences.

— The need for CNNs (2)

- A fully connected architecture is not suitable for working with images since it adds up to a huge number of parameters.
- Convolutional neural networks (CNNs) were conceptualized to overcome the difficulties, mainly working with images.
- CNNs employ weight sharing: The same set of weights is used across different spatial locations within a layer. This helps reduce the number of parameters, and facilitates feature learning.

— The need for CNNs (3)

- CNNs attempt to capture the spatial and temporal structures, which are intrinsic features of an image.
- CNNs minimize the number of parameters by using convolutional filter.
- CNNs learn hierarchical representations of features, starting from low-level features like edges and gradually learning more complex features, such as object parts and whole objects.

Overview

- Convolutional Neural Networks (CNNs) are a type of deep learning model designed specifically for processing grid-like data, such as images, but they can also be applied to other types of structured data.
- CNNs have been particularly successful in tasks related to computer vision, such as image classification, object detection, and segmentation.
- A convolutional layer is essentially a set of shared-weight perceptrons applied locally (not globally connected), making CNNs efficient for spatial data.

Types of layers

- Convolution layer: extract features from the input images.
- CNNs use convolutional layers to process and extract features from input data.
- These layers apply convolution operations to input images or feature maps.
- Convolutional operations involve sliding small filters (also called kernels) over the input to detect patterns or features like edges, textures, and shapes.

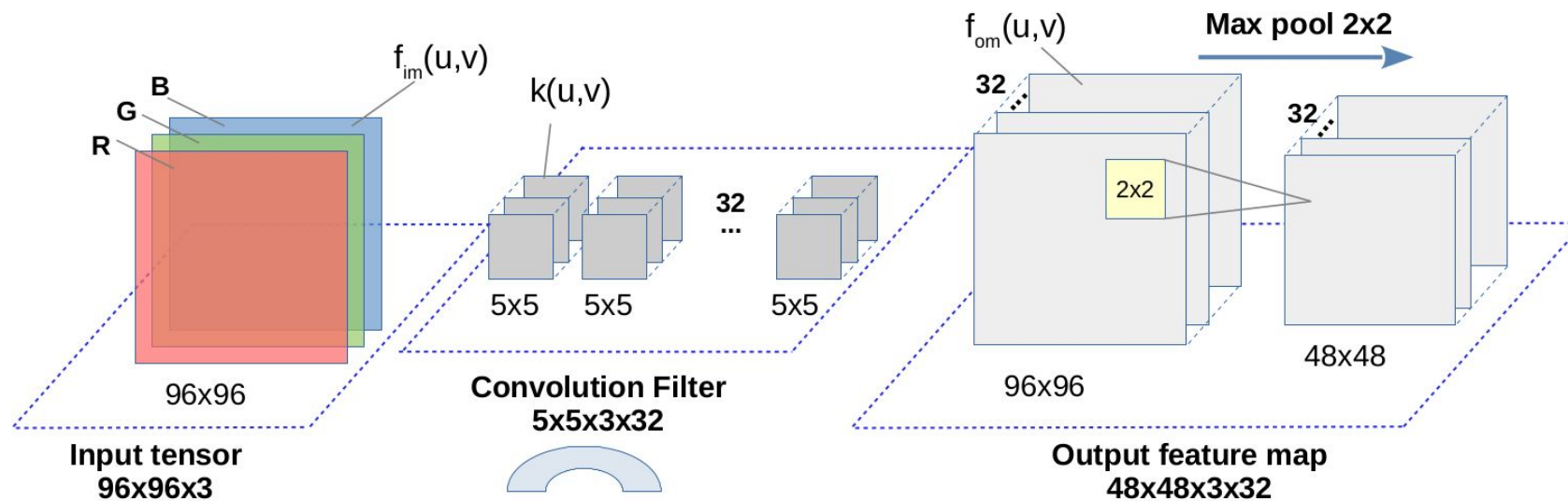
Perceptrons and feature maps

- A perceptron takes multiple inputs (features), computes a weighted sum, applies a bias, and passes it through an activation function to produce an output (e.g., 0 or 1 for classification).
- Each perceptron gets input from a certain part from the previous layer, which is called a receptive field.
- Instead of fully connected perceptrons, CNNs use convolutional layers where learnable filters (kernels) slide over input data to detect local patterns (e.g., edges in images).
- The output is a feature map, a grid of activations highlighting detected features.

Perceptrons and feature maps

- A perceptron takes multiple inputs (features), computes a weighted sum, applies a bias, and passes it through an activation function to produce an output (e.g., 0 or 1 for classification).
- Each perceptron gets input from a certain part from the previous layer, which is called a receptive field.
- Instead of fully connected perceptrons, CNNs use convolutional layers where learnable filters (kernels) slide over input data to detect local patterns (e.g., edges in images).
- The output is a feature map, a grid of activations highlighting detected features.

Filters



- A filter or kernel is a small square matrix used to capture specific features from an input image, e.g., nodes and edges.
- Several kernels can be used to extract various types of features from images.

Convolution operation

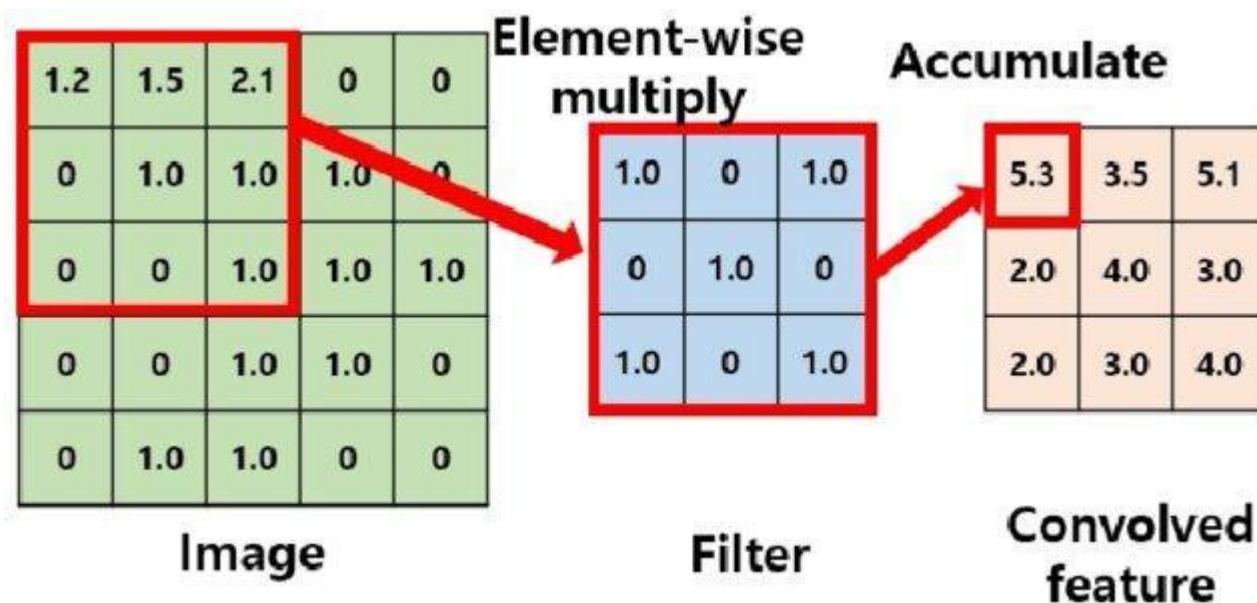


Image source:

<http://dx.doi.org/10.1109/ACCESS.2021.3122818>

<https://doi.org/10.1109/ACCESS.2021.3122818>

- The convolution operation is used to extract features from input images.
- This aims to keep important information, while still maintaining efficiency.

Pooling

- Pooling layers downsample the spatial dimensions of feature maps, reducing the amount of computation and the risk of overfitting.
- It acts as a “summarizer” for local regions, making CNNs computationally efficient and robust to small translations in input data (e.g., slight shifts in an image or code token sequence).
- Common pooling operations include max-pooling and average-pooling, which retain the most prominent values or averages in a local region.
- Building on our discussion of feature maps (where convolutions extract features), pooling helps prevent overfitting and speeds up training.

Fully-Connected Layers

- At the end of a CNN architecture, one or more fully connected layers are typically used to make predictions.
- These layers take flattened feature maps and generate output.
- In image classification tasks, the final fully connected layer often produces class probabilities using softmax activation.
- They capture interactions across all features (e.g., combining code syntax patterns for vulnerability classification).
- Dense connections scale quadratically ($O(m^2)$), making them inefficient for large inputs. This is why transformers, such as GraphCodeBERT, often replace them with attention.

Sliding

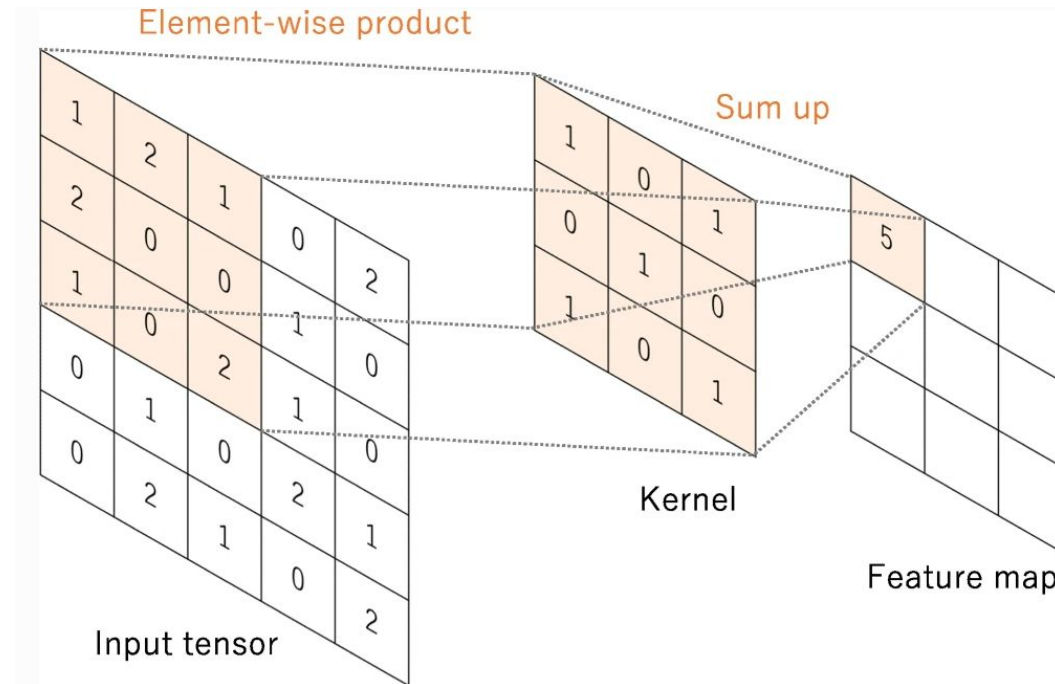


Image source: <https://insightsimaging.springeropen.com/articles/10.1007/s13244-018-0639-9>

- Convolution is performed by *sliding the filter along the width and height of a feature map*.
- Sliding (or the “sliding window/kernel mechanism”) is the process by which a learnable filter (kernel) moves across the input data (e.g., an image or sequence) to compute feature maps.
- An output feature map of one layer becomes the input feature map of the succeeding layer.

Sliding

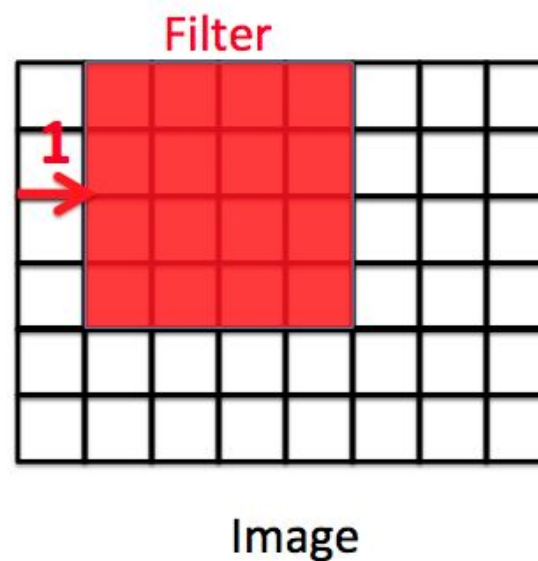
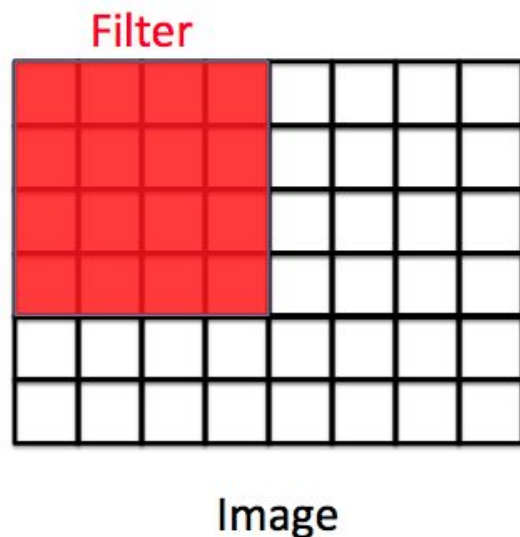


Image source: <https://medium.com/machine-learning-algorithms/what-is-stride-in-convolutional-neural-network-e3b4ae9baedb>

- Stride: The distance between two consecutive kernel positions
- A stride of 1 is normally chosen in practice (A different number is also possible).

Example of a convolved image

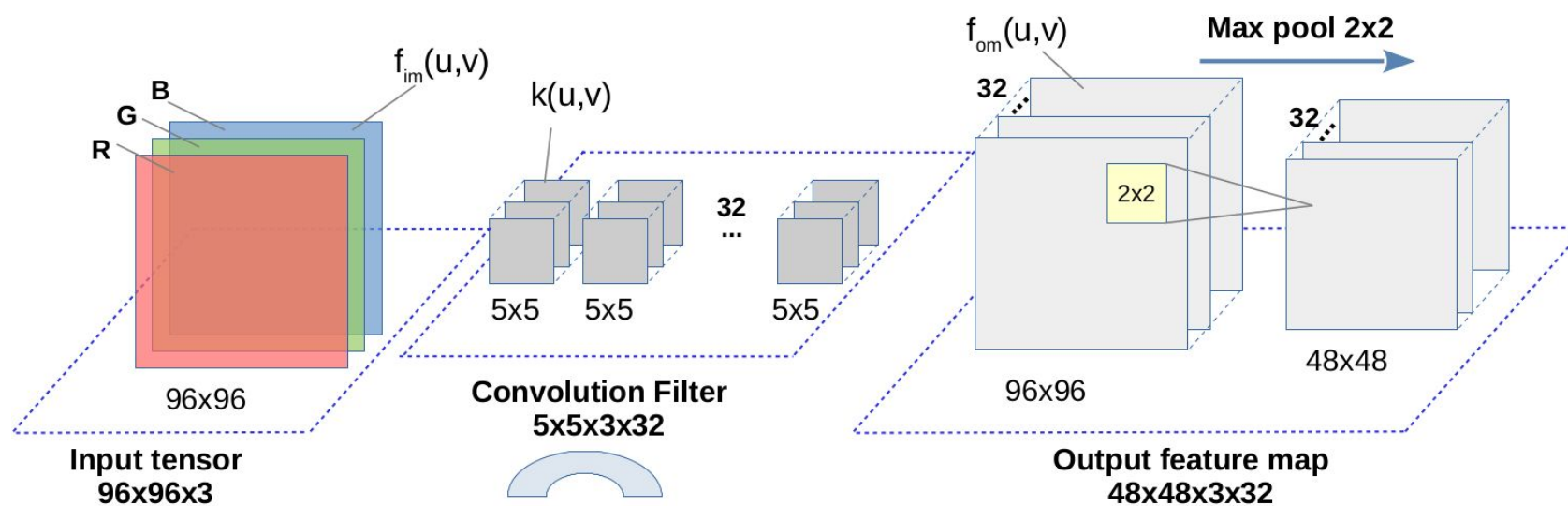


Figure 16: Convolutional layer.

Image source: <https://pub.towardsai.net/convolutional-neural-networks-cnns-tutorial-with-python-417c29f0403f>

- A convolved image stores the most intrinsic features.
- The features are useful for the classification.

Convolution layers



- Perform most of the computation within a CNN to extract important features of an input image.
- A CNN can contain several convolution layers.

Pooling layer

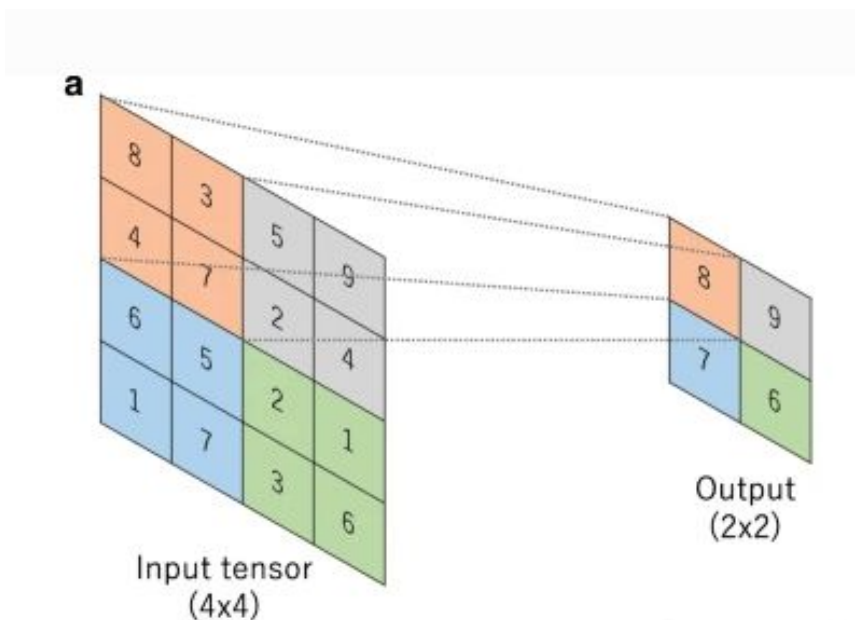


Image source: <https://insightsimaging.springeropen.com/articles/10.1007/s13244-018-0639-9>

- Scales down a feature map by taking the maximum value within a window, preferably a square ones.
- Extracts rotational and positional invariant features from a feature map.
- The ultimate aim is to reduce the number of parameters.

Pooling layer

- Pooling reduces the dimensionality of the feature maps while retaining the most important information.
- It makes the model more efficient by decreasing the number of parameters and reduces overfitting.
- Max Pooling: One of the most common types of pooling, where the maximum value from a small region of the feature map is taken.
- Average Pooling: Instead of the maximum value, the average of the values in the region is taken.

Max pooling

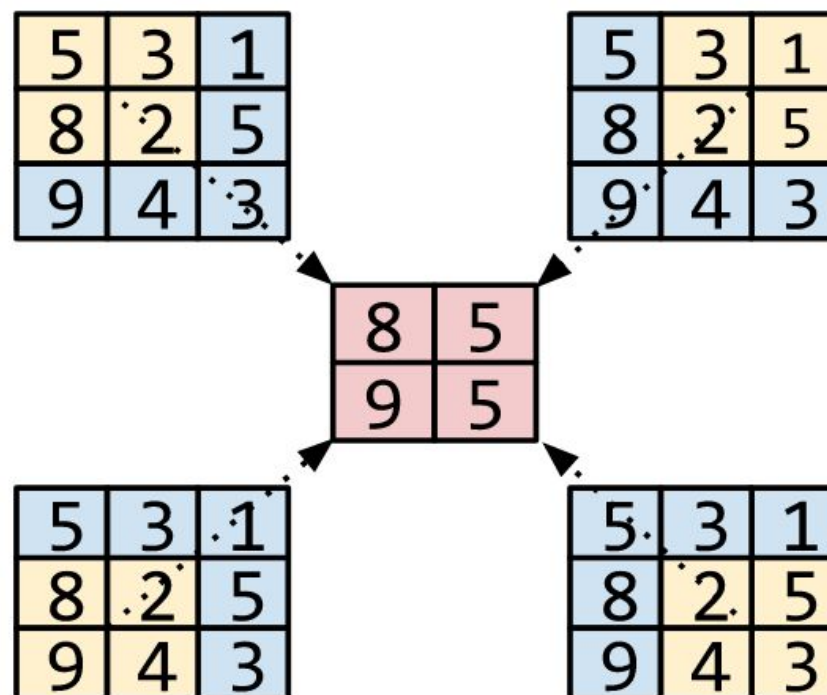


Image source: <https://developers.google.com/machine-learning/glossary>

- The maximum value within the window is selected.
- The window is shifted along the image to create the resulting feature map.

Max and average pooling

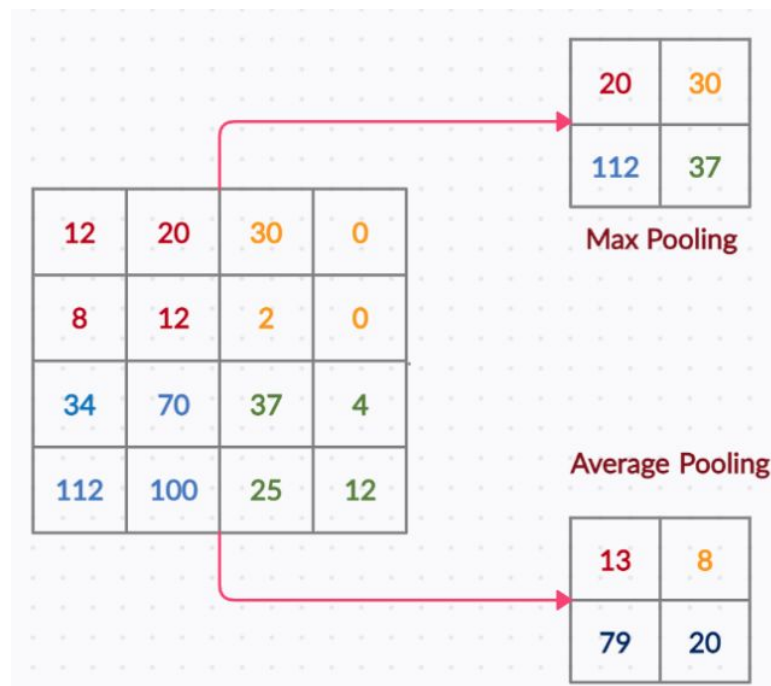


Image source: <https://pub.towardsai.net/convolutional-neural-networks-cnns-tutorial-with-python-417c29f0403f>

- Pooling can be computed as by averaging out instead of taking the maximum.

Convolutional Operations

- Performing the sliding window computation to extract hierarchical features from input data (e.g., images, sequences).
- This is done to replace the dense connections of perceptrons with local, shared-weight filters, enabling efficient pattern detection (e.g., edges in images or syntax in code).
- Convolutions generate feature maps, which are then processed by pooling or fully connected (FC) layers.
- This operation is what makes CNNs superior for spatial/temporal data, reducing parameters from $O(n^2)$ in FC layers to $O(k^2)$ per filter (k = kernel size).

Filter and convolution

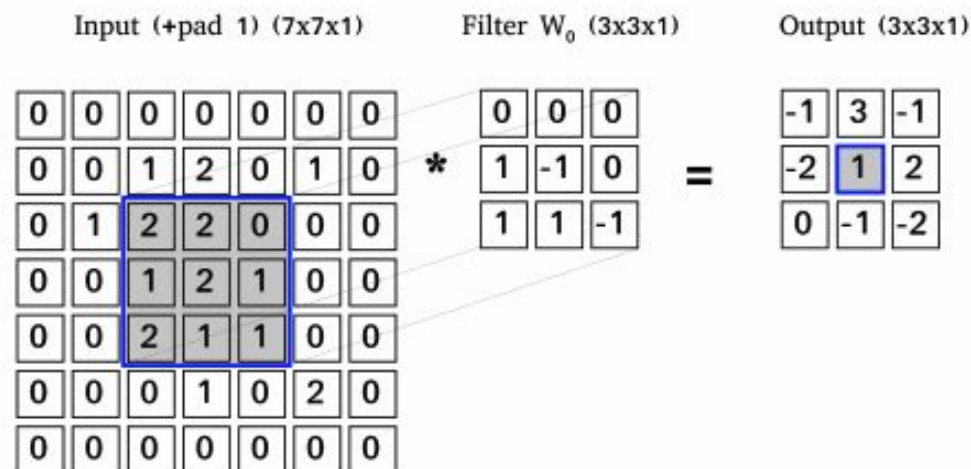


Image source: <https://www.cosmos.esa.int/web/machine-learning-group/convolutional-neural-networks-introduction>

- Convolution: $(2 \times 0 + 2 \times 0 + 0 \times 0) + (1 \times 1 + 2 \times (-1) + 1 \times 0) + (2 \times 1 + 1 \times 1 + 1 \times (-1)) = 0 + (-1) + 2 = 1$

Filter and convolution

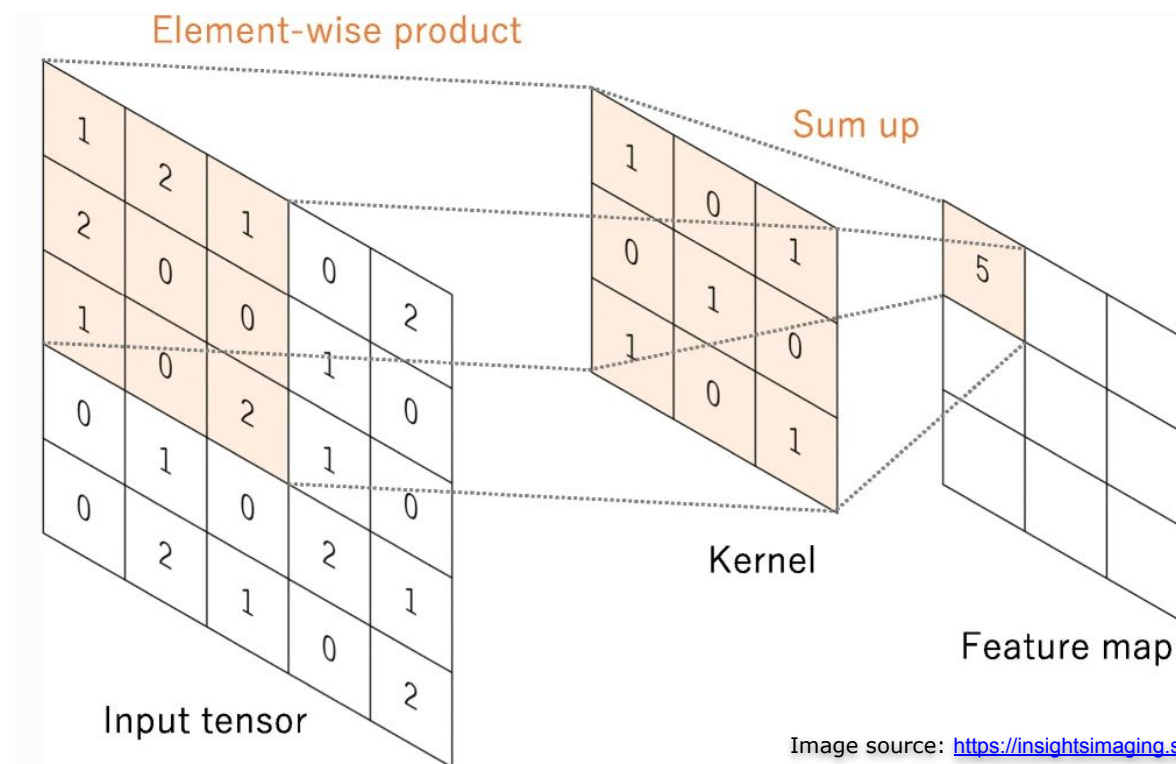


Image source: <https://insightsimaging.springeropen.com/articles/10.1007/s13244-018-0639-9>

- Convolution: $(1 \times 1 + 2 \times 0 + 1 \times 1) + (2 \times 0 + 0 \times 1 + 0 \times 0) + (1 \times 1 + 0 \times 0 + 2 \times 1) = 5$

Filter and convolution (2)

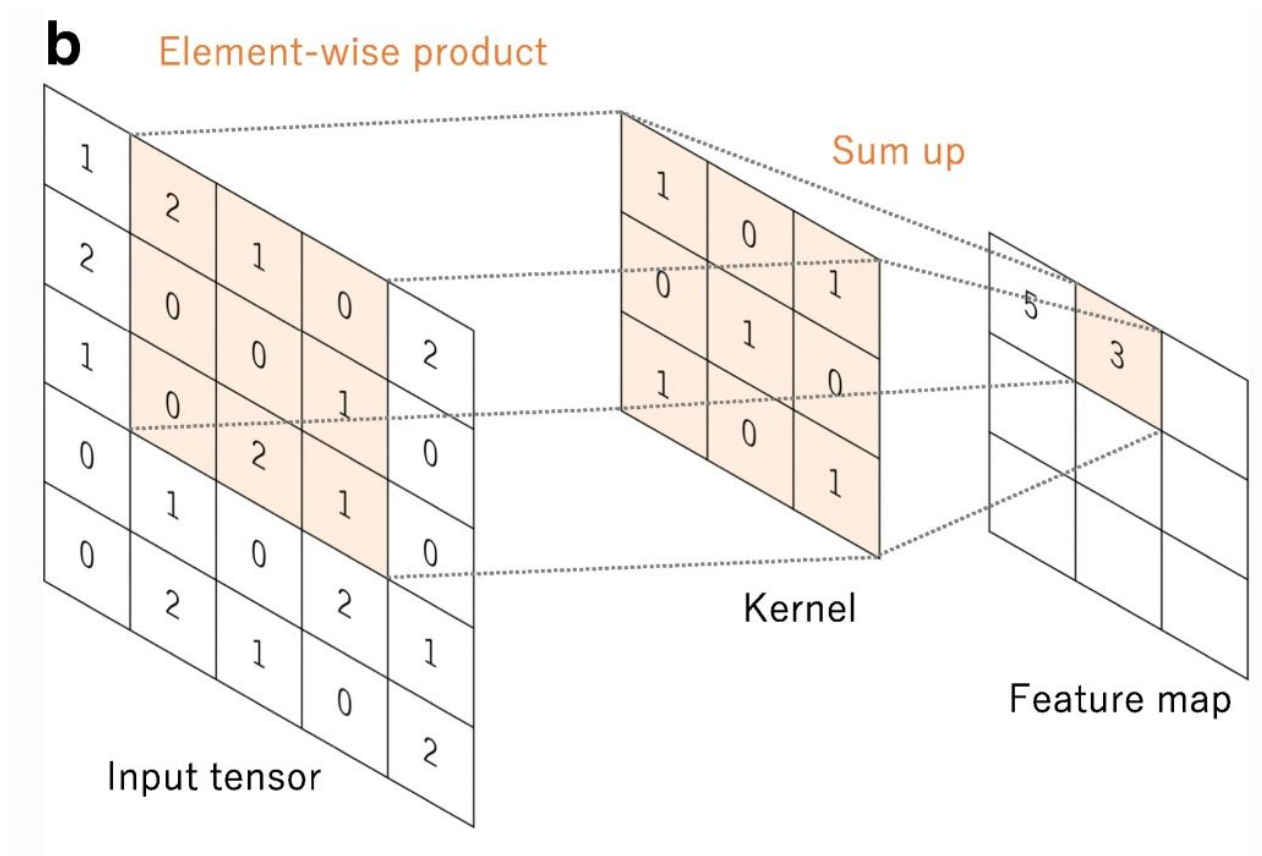


Image source: <https://insightsimaging.springeropen.com/articles/10.1007/s13244-018-0639-9>

Filter and convolution (3)

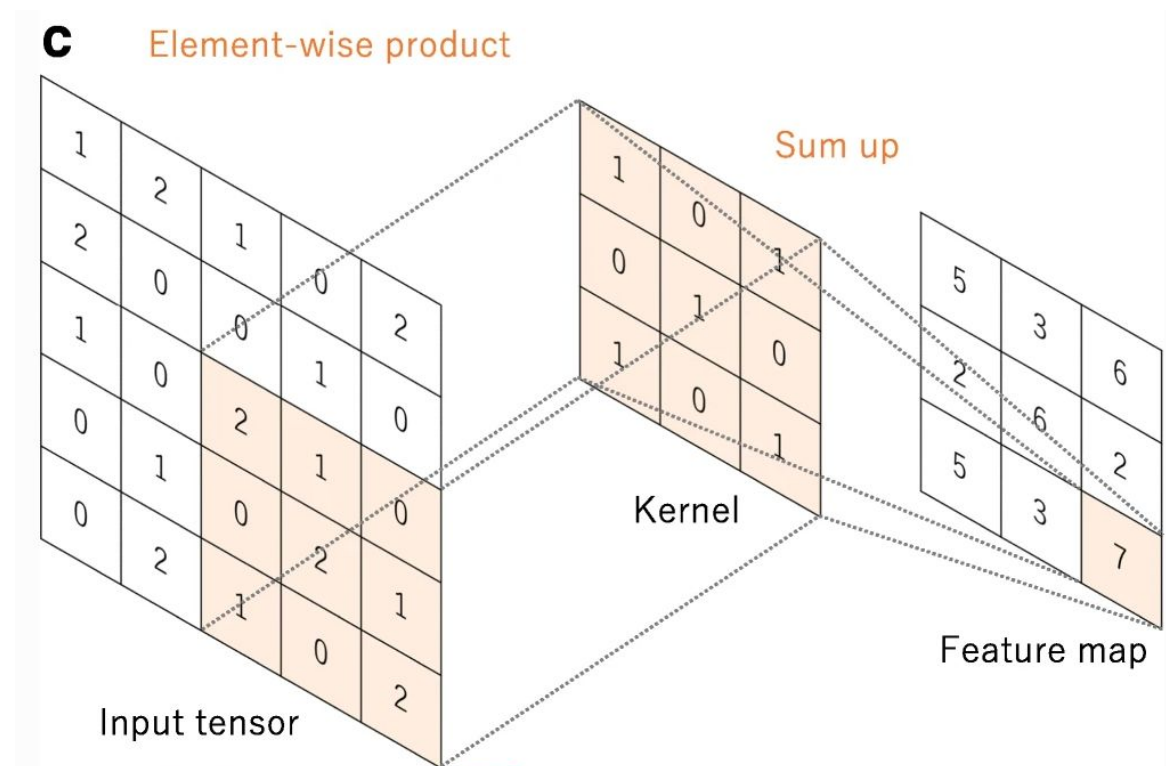


Image source: <https://insightsimaging.springeropen.com/articles/10.1007/s13244-018-0639-9>

Animation for CNNs

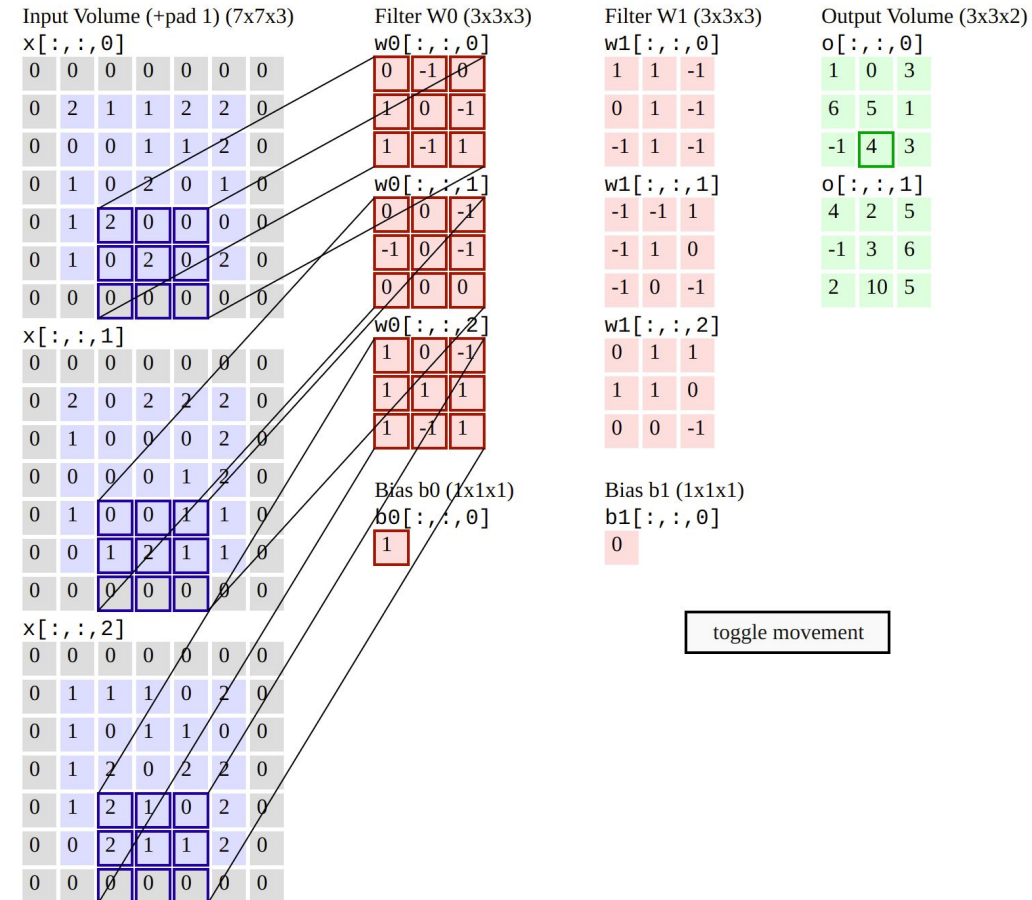


Image source: <https://cs231n.github.io/convolutional-networks/>

Fully-connected Layers

- After several convolutional and pooling layers, the output is flattened into a single vector and passed to fully connected layers, similar to traditional neural networks.
- They take the spatially extracted features from convolutional, pooling, and sliding operations and integrate them globally for final predictions.
- They function like multi-layer perceptrons (MLPs), with every neuron connected to every input, enabling the model to learn complex, non-local relationships.
- While conv layers focus on local patterns, FC layers provide the “big picture” reasoning, e.g., combining edge detections into “object” classifications.
- This layer is responsible for making the final classification or prediction.

Dropout

- Overfitting: a model performs well on the training data but does not generalize effectively to unseen data
- Dropout is a regularization method to improve the generalization of the model
- Normally, dropout happens at the fully-connected layers

Dropout

- A random subset of neurons in a layer is “dropped out” or temporarily ignored: Their outputs are set to zero, and they do not participate in the forward or backward pass of that training iteration.
- During each training iteration, neurons are dropped out with a probability defined by the dropout rate, normally set to something between 0.2 and 0.5.

Activation function

- Rectified Linear Unit (ReLU):

$$f(x) = \max(0, x)$$

- The output of the ReLU function is the input value if it's positive, and zero if it's negative.
- This creates a piecewise linear function where negative values are “rectified” to zero, and positive values remain unchanged.
- Used in the hidden (middle) layers.

Adam

- Adaptive Moment Estimation: an optimization algorithm commonly used in training convolutional neural networks.
- It adapts the learning rate for each parameter during training, maintaining a separate learning rate for each parameter, which is adjusted over time based on the past gradients of that parameter.
- Suitable for problems with large datasets and high-dimensional parameter spaces.

Regularizers

- Used to prevent overfitting.
- Improve the model's ability to generalize by reducing its complexity and overreliance on the training data.
- Common regularization techniques used in CNNs and other machine learning models include L1 and L2 regularization (Lasso and Ridge), dropout, early stopping, weight decay.
- The choice of which regularization technique to use depends on the specific problem, model architecture, and the nature of the data.

— Versions of CNNs

- LeNet-5: One of the earliest CNNs, designed for handwritten digit recognition.
- AlexNet: A breakthrough in deep learning that won the ImageNet competition in 2012.
- VGGNet: Known for its simplicity and strong performance on ImageNet.

— Versions of CNNs

- GoogLeNet (Inception): Introduced the inception module for efficient and deep networks.
- ResNet: Known for its deep residual learning approach and ability to train very deep networks.
- MobileNet: Designed for mobile and embedded vision applications, balancing accuracy and speed.

Advantages

- CNNs are able to learn hierarchical features. Early layers capture low-level features (e.g., edges), while deeper layers capture more abstract features (e.g., shapes or objects).
- The use of filters allows CNNs to share parameters across the input, making the model more efficient and reducing the number of parameters compared to fully connected networks.
- CNNs are good at recognizing objects or patterns, regardless of their position in the input, thanks to their convolutional nature.

Applications

- CNNs are widely used in tasks like identifying objects, faces, or animals in images (e.g., ImageNet competition).
- CNNs power object detection models such as YOLO (You Only Look Once) and Faster R-CNN, where the goal is to not only classify objects but also locate them within an image.
- CNN-based models like U-Net and Fully Convolutional Networks (FCNs) are used to segment images by labeling each pixel according to its object category.

Applications

- Natural Language Processing (NLP): CNNs are sometimes used in NLP tasks such as sentence classification or document categorization, though recurrent networks like LSTMs are more commonly used.
- Medical Imaging: CNNs are used in medical diagnosis, such as detecting tumors in X-rays or MRI scans.

Limitations

- CNNs generally require large amounts of labeled data for training, which can be a challenge in domains with limited data.
- Training CNNs, especially deep ones, can be computationally expensive and require powerful GPUs.
- CNNs excel at capturing local patterns, but sometimes fail to capture long-range dependencies unless specially designed (e.g., using dilated convolutions or attention mechanisms).